---

1. Regras de Negócio (O "O quê")

Para deixar o sistema robusto, implemente estas regras no seu `Service`:

- **Limite de Empréstimos:** Um usuário não pode ter mais de 3 livros simultâneos.
- **Sistema de Multas:** Se o livro atrasar, o usuário fica "bloqueado" para novos empréstimos até devolver o pendente e pagar uma multa (ex: R$ 2,00 por dia).
- **Gestão de Estoque Real:** Você não "deleta" um livro. Você diminui a `quantidade_disponivel`. Se a quantidade for 0, o status do livro muda para `ESGOTADO`.
- **Reserva de Livros:** Se um livro está esgotado, o usuário pode entrar numa fila de reserva. Quando o livro for devolvido, o sistema avisa o primeiro da fila (ou reserva automaticamente por 24h).
- **Ciclo de Vida do Exemplar:** Um livro pode ser marcado como `PERDIDO` ou `DANIFICADO`, o que deve disparar um processo de baixa no estoque e cobrança do valor do livro ao usuário.

---

2. Desafios de Engenharia (O "Como")

Aqui é onde você realmente exercita o cérebro de engenheiro de software:

A. O Problema da Concorrência (Race Conditions)

Reflexão: Imagine que só resta **1 exemplar** de "Clean Code". Dois usuários clicam em "Emprestar" exatamente ao mesmo tempo (milissegundos de diferença).

- **Problema:** Se você apenas ler o banco e depois salvar, os dois podem conseguir o livro, deixando o estoque em -1.
- **Desafio:** Estude **Optimistic Locking** (`@Version` no JPA) ou **Pessimistic Locking** para garantir que apenas uma transação vença.

B. Soft Delete vs Hard Delete

Reflexão: Se você deletar um livro do banco (`DELETE FROM...`), o que acontece com o histórico de empréstimos que apontava para o ID dele? O banco explode ou os dados somem?

- **Desafio:** Implemente **Soft Delete** (uma coluna `boolean ativo` ou `deleted_at`). O dado continua lá para histórico, mas não aparece nas buscas da biblioteca.

C. Paginação e Filtros

Reflexão: E se sua biblioteca tiver 1 milhão de livros? Você vai retornar um JSON com todos? (Sua memória RAM vai chorar).

- **Desafio:** Use `Pageable` e `Slice` do Spring Data JPA. Implemente filtros dinâmicos (buscar por título, autor e categoria ao mesmo tempo).

D. Docker & Resiliência

Reflexão: No Docker, o banco de dados demora mais para subir que a aplicação Spring.

- **Desafio:** Configure um `healthcheck` no seu `docker-compose.yml` para que o Spring só tente conectar quando o PostgreSQL estiver realmente pronto para receber visitas.

---

3. Lista de Funcionalidades (Checklist)

- [ ] **Autenticação (JWT):** Diferenciar `USUARIO` de `BIBLIOTECARIO`. Só o bibliotecário cadastra livros.
- [ ] **Histórico de Empréstimos:** Uma tabela que registra quem pegou o quê, quando pegou e quando devolveu.
- [ ] **Dashboard Simples:** Um endpoint que retorna estatísticas (livro mais lido do mês, usuários inadimplentes).
- [ ] **Logs de Auditoria:** Salvar em uma tabela (ou arquivo) cada vez que um livro mudar de preço ou for removido.
- [ ] **Perfil do Usuário:** Mostrar os livros que ele já leu e recomendações baseadas na categoria.